Design Pattern Analysis in a Flask-Based Event Management Application

Team A15

Harshitha D — AM.SC.U4CSE23222
T Heshaja — AM.SC.U4CSE23265

2026-10-06

Contents

1 Application and Architecture **2**
1.1 Application . . . 2
1.2 Original Source and Environment . . . 2
1.3 Scope and Architecture . . . 2
1.4 Run and Verification . . . 3

2 Part 1 Pattern Discovery **3**
2.1 Method . . . 3
2.2 Baseline Totals . . . 3
2.3 Pattern Inventory . . . 3
2.4 Confirmed Pattern Records . . . 4
2.4.1 Event Creation MVC Flow . . . 4
2.4.2 Authenticated Event Route Registration . . . 4
2.4.3 Event and Ticket Persistence Model . . . 4
2.4.4 Event Creation Validation . . . 4
2.5 Rejected Candidates . . . 4

3 Part 2 Evolution and Refactoring **5**
3.1 CH01 — Repository Pattern . . . 5
3.2 CH02 — Strategy Pattern . . . 5
3.3 CH03 — Observer Pattern . . . 5
3.4 Updated Pattern Counts . . . 5
3.5 Before/After Metrics . . . 6

4 Part 3 Cross-Language Comparison **6**
4.1 Selected Instances and Common Specification . . . 6
4.2 Measured Results . . . 6
4.3 Static and Dynamic Comparison . . . 7

5 LLM Records and Human Verification **7**

6 Findings, Contributions and References **7**
6.1 Findings . . . 7
6.2 Limitations . . . 7
6.3 Team Contributions . . . 8
6.4 References . . . 8

7 Team Folder and Dataset Map **8**

1 Application and Architecture

1.1 Application

The selected application is an Event Management System implemented as a Python/Flask web application. It supports organizer and attendee workflows including authentication, event creation and management, event discovery, ticket registration, and ticket management. The supplied repository also contains a PHP subtree; that subtree was excluded from the case-study analysis because the assigned case is specifically the Flask-based application.

1.2 Original Source and Environment

- Source repository: `https://github.com/ARUNAGIRINATHAN-K/Event-Management-system`
- Access/inspection date: 2026-10-06.
- Baseline source: repository ZIP snapshot supplied for this case study. The ZIP did not preserve a Git commit hash, so no commit identifier is invented.
- Python requirement: $\geq$3.11.
- Locked dependency versions: Flask 3.1.0, Flask-Login 0.6.3, Flask-SQLAlchemy 3.1.1, Flask-WTF 1.2.2, SQLAlchemy 2.0.40.
- Persistence: SQLAlchemy ORM; the application uses a configured database URL with a SQLite fallback in the source.

1.3 Scope and Architecture

The inspected baseline scope includes `app.py`, `models.py`, `forms.py`, `routes.py`, `utils.py`, templates, static assets, and configuration/dependency files. The PHP subtree was excluded. The baseline request path for event creation is a controller route that validates an `EventForm`, constructs an `Event`, writes through `db.session`, and renders or redirects through the template layer.

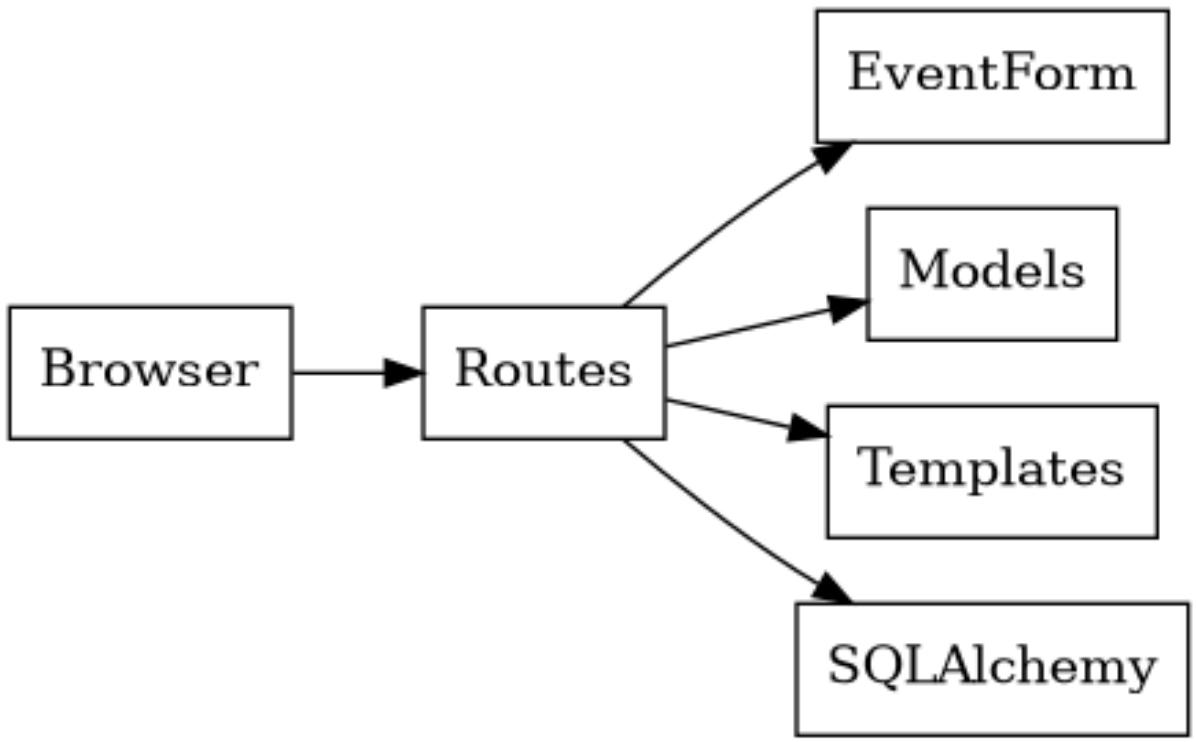


Figure 1: Baseline Flask application architecture.

After refactoring, the route layer delegates persistence to repositories, filtering to strategies, and registration side effects to an observer-enabled registration service.

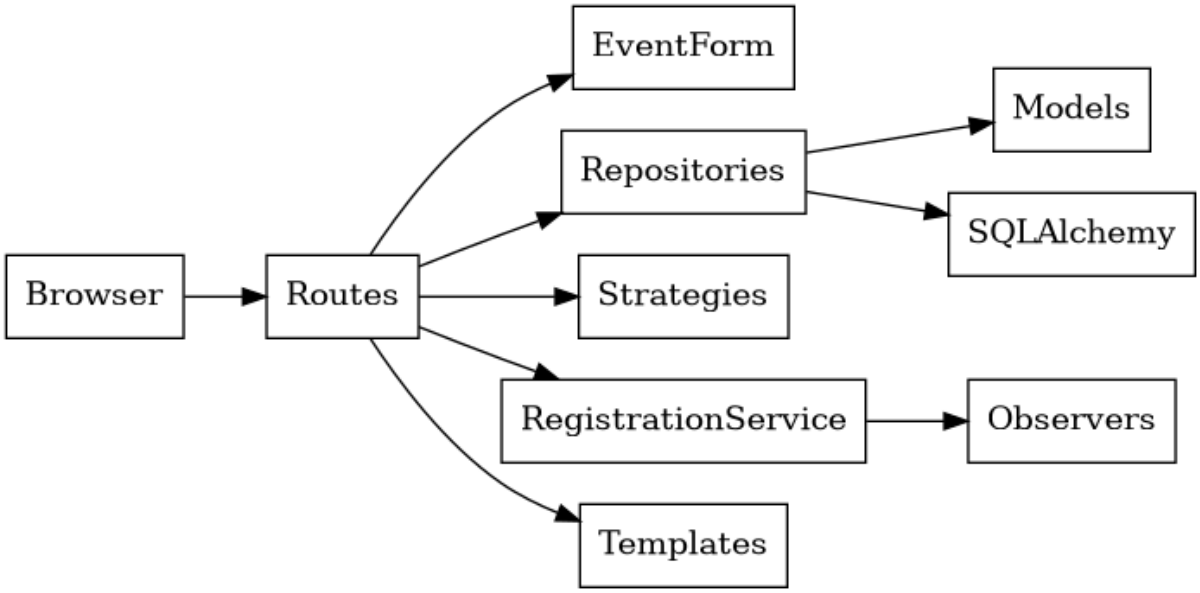


Figure 2: Evolved architecture after CH01–CH03.

1.4 Run and Verification

The supplied project declares its dependencies in `pyproject.toml` and locks versions in `uv.lock`. The structural pattern test command used for the refactored source was:

```
python3 tests/test_design_patterns.py
```

Actual result: 4 tests passed. The full Flask application was not claimed as runtime-tested in the verification container because the Flask runtime dependency was unavailable there; this limitation is preserved rather than replaced with an estimated result.

2 Part 1 Pattern Discovery

2.1 Method

Candidates were identified from source structure and behavior, then checked against actual collaborations: participants, calls/delegation, responsibilities, and evidence in source. A class name, decorator annotation, or LLM assertion alone was not treated as sufficient evidence. Candidate patterns that lacked the required collaboration were rejected.

2.2 Baseline Totals

- Confirmed pattern types: **4**.
- Confirmed pattern instances: **4** independent collaborations.
- Concrete participant classes: **3** distinct classes across the baseline records (`EventForm`, `Event`, `Ticket`); functions, decorators, templates, and objects are not counted as classes.
- Unique participant symbols: **12** after deduplication across the four baseline records.
- Rejected candidates: Singleton, Strategy, Observer, Facade.
- Uncertain candidates: none retained as confirmed baseline patterns.

2.3 Pattern Inventory

Pattern	Instance	Type	Instances
MVC	Event Creation MVC Flow	Architectural	1

Decorator	Authenticated Event Route Registration	Framework idiom	1
Active Record-like ORM	Event and Ticket Persistence Model	Architectural/Data Access	1
Form Object	Event Creation Validation	Framework/Application idiom	1

2.4 Confirmed Pattern Records

2.4.1 Event Creation MVC Flow

Pattern/category: MVC / architectural.
Origin: Application architecture.
Participants: controller = `routes.register_routes.create_event`; form = `forms.EventForm`; model = `models.Event`; view = `templates/organizer/create_event.html`; persistence = `db.session`.
Evidence: the controller creates and validates the form, constructs an Event, commits it through the SQLAlchemy session, then redirects or renders the template. Baseline source locations are `routes.py:118--149`, `forms.py:43--61`, and `models.py:50--86`.
Verification: confirmed from source behavior; no separate MVC framework class hierarchy was invented.

2.4.2 Authenticated Event Route Registration

Pattern/category: Decorator / framework idiom.
Participants: Flask route decorator, Flask-Login `login_required`, and decorated `logout` function.
Evidence: `routes.py:95--100` shows `@app.route('/logout')` and `@login_required` wrapping the route function. The authentication behavior is added around the function without changing its body. This is reported as a framework/Python decorator idiom rather than a complete GoF Decorator subsystem.
Verification: confirmed.

2.4.3 Event and Ticket Persistence Model

Pattern/category: Active Record-like ORM / architectural-data access.
Participants: `models.Event`, `models.Ticket`, SQLAlchemy `db`, and route-level query/session operations.
Evidence: Event and Ticket combine persistent fields with domain behavior while the baseline routes directly query, create, and commit these objects. Key locations are `models.py:50--114`, `app.py:17--33`, `routes.py:130--144`, and `routes.py:328--371`.
Verification: confirmed as an Active Record-like ORM collaboration; it is not claimed to be a pure textbook Active Record implementation.

2.4.4 Event Creation Validation

Pattern/category: Form Object / framework-application idiom.
Participants: `forms.EventForm`, its date validators, the event creation controller, and the event creation template.
Evidence: `EventForm` encapsulates event fields and cross-field validation; the controller calls `validate_on_submit(` before creating the model. Locations include `forms.py:43--61` and `routes.py:118--149`.
Verification: confirmed.

2.5 Rejected Candidates

- **Singleton:** module-level Flask/SQLAlchemy objects are not evidence of an explicit Singleton implementation with controlled construction/access.

- **Strategy:** baseline event filtering is implemented directly with conditional query modifications in `routes.py:275--291`; there are no interchangeable strategy objects.
- **Observer:** baseline registration creates and commits tickets directly and has no subject/observer subscription collaboration.
- **Facade:** utility functions did not provide sufficiently clear evidence of one unified subsystem facade, so the candidate was not counted.

3 Part 2 Evolution and Refactoring

Each change was implemented as a real code modification and represented as a new collaboration only when the source showed the corresponding participants and behavior. The final evolution CSV preserves the baseline rows and appends the post-change snapshots.

3.1 CH01 — Repository Pattern

Requirement: remove direct persistence/query responsibilities from Flask route handlers.
Before: routes directly used `Event.query`, `Ticket.query`, `db.session.add`, and `db.session.commit`.
After: `EventRepository`, `TicketRepository`, and `UserRepository` encapsulate query and persistence operations. Routes obtain domain objects through repository methods.
Pattern effect: Repository — newly introduced. The existing MVC and ORM collaborations are modified because persistence is now mediated by repositories.
Verification: structural test suite 4/4 passed; modified Python files compile.
Refactoring rationale: the route module mixed HTTP/controller responsibilities with persistence mechanics, creating a separation-of-concerns problem.

3.2 CH02 — Strategy Pattern

Requirement: make event filtering criteria interchangeable.
Before: the attendee event route contained separate conditional query filters for text, category, start date and end date.
After: `EventFilterStrategy` defines the strategy contract; text, category and date-range strategies implement it. The route selects active strategies and `EventRepository.search_upcoming()` applies them sequentially.
Pattern effect: Strategy — newly introduced.
Verification: structural tests and Python compilation passed.
Refactoring rationale: the route's conditional filtering logic was a direct collection of policy branches; strategies isolate each criterion and permit additional criteria without rewriting the repository's application loop.

3.3 CH03 — Observer Pattern

Requirement: decouple registration side effects from the registration route.
Before: the route created and committed tickets itself.
After: `EventRegistrationService` acts as the subject. It creates tickets through `TicketRepository`, then notifies attached `RegistrationNotificationObserver` and `RegistrationStatisticsObserver` instances.
Pattern effect: Observer — newly introduced.
Verification: structural tests and Python compilation passed.
Refactoring rationale: ticket creation and post-registration reactions are separated so new reactions can be attached without adding more side-effect code to the route.

3.4 Updated Pattern Counts

Snapshot	Pattern types	Pattern instances	Newly introduced
Original	4	4	–
After CH01	5	5	Repository
After CH02	6	6	Strategy
After CH03	7	7	Observer

3.5 Before/After Metrics

Using the same source-LOC rule (nonblank, non-comment production lines), `routes.py` decreased from 340 LOC to 336 LOC. The new repository, strategy, observer and service modules add 193 nonblank, non-comment lines. This demonstrates responsibility movement rather than a claim that fewer total lines automatically means better design. Complexity/timing for the small cross-language snippets is recorded separately in `data/cross_language.csv`.

4 Part 3 Cross-Language Comparison

4.1 Selected Instances and Common Specification

The three newly introduced patterns were selected because they represent the actual Part 2 refactorings. Each was reduced to a small deterministic problem and implemented in Java, Python, JavaScript and C++. The same conceptual inputs and expected outputs were used within each pattern family.

- **Repository:** store events and retrieve an event by identifier.
- **Strategy:** filter a common event set using interchangeable name/category criteria.
- **Observer:** register an attendee and produce both notification and statistics reactions.

Compilation/interpreter versions, commands, source LOC, maximum function cyclomatic complexity, test result, median process time and language notes are stored in `data/cross_language.csv`. The timing protocol was compile first, one warmup, five measured process runs, then median. Timing includes process startup and test execution.

4.2 Measured Results

Pattern	Language	LOC	Max CC	Median ms
Repository	Java	3	1	24.629
Repository	Python	11	1	594.444
Repository	JavaScript	8	1	21.222
Repository	C++	6	1	1.105
Strategy	Java	6	2	26.806
Strategy	Python	13	1	567.115
Strategy	JavaScript	5	1	21.456
Strategy	C++	11	2	1.117
Observer	Java	6	1	56.745
Observer	Python	15	1	571.284
Observer	JavaScript	4	1	22.141
Observer	C++	9	1	1.356

All 12 implementations passed their tests. These values are process-level measurements on the same verification machine, not claims about the intrinsic performance of the languages or of the original Flask application.

4.3 Static and Dynamic Comparison

Java uses explicit interfaces/classes for the Strategy and Observer collaborations. Python uses lightweight classes and duck-typed collaboration, resulting in compact implementations. JavaScript uses object/-class composition with less boilerplate. C++ uses explicit classes and compilation with C++17. The maximum function complexity remained low (1–2) because the representative problems were intentionally small. Runtime differences are dominated by process startup, interpreter/JVM initialization and the small workload, so they should not be interpreted as production performance rankings.

5 LLM Records and Human Verification

The project records three major AI-assisted analysis interactions in `data/llm.csv`, with prompt, output and verification files under the corresponding Part1/Part2/Part3 folders. The model displayed for these records was ChatGPT GPT-5.6 Luna on 2026-10-06.

Part	Analysis	Evaluation	Verification
Part 1	Baseline pattern inventory	Correct after source-based checking; unsupported candidates rejected	Source locations and collaboration behavior in baseline Flask code
Part 2	Refactoring/evolution	Correct for implemented changes; runtime limitation explicitly retained	4/4 structural tests and Python compilation
Part 3	Cross-language analysis	Correct	12/12 implementations passed; five-run median measurements

The evidence files explicitly distinguish reconstructed/conversation-derived prompt context from verbatim historical files where applicable. No invented LLM experiment is presented as an actual run.

6 Findings, Contributions and References

6.1 Findings

1. The baseline Flask application already contains several recognizable architectural/framework idioms, but not every common GoF pattern is present. In particular, Strategy and Observer were not counted until concrete collaborations were introduced.
2. CH01 separates HTTP/controller concerns from persistence mechanics without removing the underlying SQLAlchemy model layer.
3. CH02 converts hard-coded filtering branches into independently selectable strategies.
4. CH03 separates registration side effects from the route and provides explicit extension points for new reactions.
5. Cross-language structure varies substantially in boilerplate and runtime behavior, while the representative pattern responsibilities remain comparable.
6. Source LOC alone is insufficient to establish a refactoring as better; the report therefore combines responsibility placement, pattern collaboration and test evidence with LOC measurements.

6.2 Limitations

The baseline Git commit hash was not preserved in the supplied ZIP. Full Flask runtime execution was not available in the verification container, so only the actual structural/compilation result is reported. Cross-language timing is process-level and based on small deterministic snippets; it is not a benchmark of production systems. Individual team-member task allocation was not included in the supplied case-study materials and is therefore not invented.

6.3 Team Contributions

The two listed team members jointly constitute Team A15. The supplied materials identify membership but do not provide individual task allocation; consequently, the submission records team-level work rather than assigning unsupported individual roles.

6.4 References

1. 23CSE455 Design Pattern Case Study Instructions, supplied course PDF, accessed 2026-10-06.
2. 23CSE455 Design Patterns Case Study Report Template, supplied course PDF, accessed 2026-10-06.
3. Event Management System repository: `https://github.com/ARUNAGIRINATHAN-K/Event-Management-syst`
4. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 2000.
5. Flask documentation: `https://flask.palletsprojects.com/`.
6. SQLAlchemy documentation: `https://docs.sqlalchemy.org/`.

7 Team Folder and Dataset Map

The final submission folder is `A15/`. It contains one report, one dataset directory, and supporting Part1/Part2/Part3 evidence. The five required CSV files occur only in `data/`.

```
A15/
  README.md
  A15_Report.pdf
  A15_Report.tex
  data/
    patterns.csv
    pattern_counts.csv
    patterns_evolution.csv
    cross_language.csv
    llm.csv
  code/
    original/
    architecture/
  Part1/
    code/
    evidence/
    llm/
  Part2/
    code/evolved/
    changes/Change1..Change3/
    evidence/
    llm/
  Part3/
    Repository/
    Strategy/
    Observer/
    test-cases/
    evidence/
    llm/
```